Visual Surround Suppression using Excitation-Inhibition Circuits

NX-465 Mini-project **MP1**

Spring semester 2026

* Read the general instructions carefully before starting the mini-project. *

Introduction

Neurons in the brain can be broadly grouped into two categories based on their interaction with other neurons. A neuron can either *excite* or *inhibit* other neurons. While this may seem simple at first, it actually places strong constraints on how neural circuits can operate. A functional network requires stability between the two types: without excitation there would be no activity, and without inhibition there would be no mechanism to regulate or reduce activity.

In this project you will implement a field model of the cortical sheet that has been used to explain a counter-intuitive physiological effect observed in experiments. The model explicitly includes both excitatory and inhibitory neuronal populations. You will begin by implementing two interacting populations of Leaky Integrate-and-Fire neurons and studying the stability and general dynamical properties of their interaction. You will then extend the model by coupling many such populations together to form a spatially structured cortical sheet, allowing you to explore the emergence of localized "bumps" of activity. Finally, you will examine how this network responds to external stimuli and reproduce the paradoxical effect observed experimentally.

Note: the project is intended to be solved using Python without the need for any specific library (other than the usual `numpy`, `scipy` and `matplotlib`). You are free to use other libraries if you want.
At the bottom of the project you will find a list of resources and references for further reading.

Ex 0. Getting Started: Population of LIF neurons

This project will be using Leaky Integrate-and-Fire neurons. We start by implementing a population of unconnected LIF neurons.

$$\tau_m \frac{du_i(t)}{dt} = -u_i(t) + RI_i(t) \tag{1}$$

when the potential crosses the threshold θ, the neuron emits a spike and the potential is reset to u_{reset}. The spike train of neuron i is denoted as $s_i(t) = \sum_f \delta(t - t_i^f)$ with t^f the spiking times of the neuron and δ the dirac delta function.

Neuron parameters: We will be using the parameters $\theta = 20$ mV, $u_{\text{reset}} = -10$ mV, $R = 1\text{M}\Omega$, $\Delta t = 0.5$ ms, $\tau_m = 20$ ms.

0.1. Simulate the dynamics of $N = 100$ unconnected neurons for $T = 1000$ ms, receiving the slowly oscillating input current $I_i(t) = I_0(1 + \sin(\omega t))$ with $I_0 = 20$ nA, $\omega = 10$ rad/s, and with all neurons randomly initialised uniformly between u_{reset} and θ: $u_i(0) \sim \text{Unif}\,(u_{\text{reset}}, \theta)$.

- Make a rasterplot of the spiking times of all neurons over time. That is, make a scatterplot where each spike is a dot, with time on the x-axis and neuron id on the y-axis.
- Plot the mean firing rate over time and compare it to the external input. The mean firing rate is the number of spikes per time step across the N neurons. What do you observe?

Hints:

- Write a method that runs the evolution of the membrane potentials, according to Eq. (1). It should take as an argument the initial potentials $u_i(t = 0)$, and return all the potentials and spikes of the neurons through time.
- Eq. (1) can be integrated directly using the forward Euler method, with discrete time steps $t = t_0, t_1, \ldots$ ($t_k = k \cdot \Delta t$). The spike train becomes $s_i(t_k) = 1/\Delta t$ when the neuron spikes and zero otherwise. It gives the following update rule:

$$u_i(t_{k+1}) = u_i(t_k) + \Delta t \cdot \frac{du_i}{dt}(t_k) \tag{2}$$

As you may notice, it takes a lot of input current to make each neuron spike. We now add stochastic background current $I_i^{\text{bg}}(t)$ to each neuron, which mimics a baseline arrival of spikes coming from background neurons outside of the population. For simplicity, we model this as a Poisson-distributed input in discrete time, $I_i^{\text{bg}}(t_k) \sim 1 \text{ nA} \cdot \text{Poisson}\left(\bar{n}^{\text{bg}}\right)$, with $\bar{n}^{\text{bg}}$ the mean number of events in one time step. This background input is independently sampled for each neuron for each time step. We include this stochastic background input for all following exercises and take $\bar{n}^{\text{bg}} = 25$. The total input to a neuron i can then be written as $I_i(t) = I_i^{\text{ext}}(t) + I_i^{\text{bg}}(t)$.

0.2. Plot the f-I curve for a population of $N = 100$ neurons receiving stochastic background current. The f-I curve is defined as the mean firing rate in response to constant external current $I_i^{\text{ext}}(t) = I_0$ to all neurons.

- Take $I_0 = -10$ nA to 50 nA in steps of 5 nA. For each I_0, simulate the network for $T = 100$ ms and report the mean firing rate averaged over the last 50 ms.
- In the same plot, plot also the theoretical f-I curve for LIF neurons as derived in class (without background input; see question set 1). What are the differences that you see and where do they come from?

Tip A useful package to have in python is the `tqdm` package for creating a progress bar on your simulations:

```
from tqdm import tqdm

for step in tqdm(range(n_steps)):
    ...
```

Ex 1. Excitation-inhibition balance

Now that we have a working implementation of the LIF neurons we will group them into two populations, one excitatory and one inhibitory, and define their synaptic interactions.
For this, we add another term to the input to each neuron to include synaptic interactions,

$$I_i(t) = \underbrace{\sum_{j=1}^{N} w_{ij} s_j(t - \tau_{\text{delay}})}_{I_i^{\text{syn}}(t)} + I_i^{\text{ext}}(t) + I_i^{\text{bg}}(t) \tag{3}$$

where w_{ij} is the synaptic strength from neuron j to neuron i (in picoCoulomb; pC) and τ_{delay} is the synaptic transmission delay. $I_i^{\text{ext}}(t)$ and $I_i^{\text{bg}}(t)$ are the external stimulus and stochastic background input, respectively.
Each neuron is either excitatory (E) or inhibitory (I). For simplicity, we take all excitatory connections to be of equal strength, and similarly for inhibitory ones:

$$w^{E \leftarrow E} = w^{I \leftarrow E} = J \tag{4}$$

$$w^{E \leftarrow I} = w^{I \leftarrow I} = -gJ \tag{5}$$

We take the number of excitatory neurons (N_E) larger than the number of inhibitory ones ($N_I = \gamma N_E$), using $\gamma = 0.25$. The total number of neurons is then $N = N_E + N_I$. Next, we make the network sparse by removing for each neuron a randomly selected subset of incoming connections (i.e., weight = 0). We make it such that each neuron receives exactly $K_E = pN_E$ excitatory connections and exactly $K_I = pN_I$ inhibitory connections, where p is the connection fraction. This defines the connectivity matrix w of size $N \times N$.
Network parameters: The network parameters we will use throughout this project are $N_E = 1000$, $\gamma = 0.25$, $p = 0.02$, $g = 5$, $J = 45$ pC and $\tau_{\text{delay}} = 2$ ms.

1.1. Write a method for generating a random sparse connectivity matrix based on the constraints above, and implement it as a function `generate_sparse_connectivity(NE, NI, KE, KI, J, g)`.
Hint: As each neuron only receives few incoming connections, it is much more computationally efficient to implement this using a sparse matrix, for example using the `csr_matrix` from `scipy`:

```python
from scipy.sparse import csr_matrix

w = ... # generate the full matrix
w = csr_matrix(w) # convert to a sparse matrix
```

With this, we are now ready to implement our network of the two interacting populations.

1.2. Implement the network as described above. Plot the f-I curves for the excitatory and inhibitory population in the same plot, using the same procedure as outlined in Ex. 0.2. For each I_0, extract the mean firing rates of both the excitatory and inhibitory populations, respectively, and use this to construct your two f-I curves.

The network is said to be in the *balanced state* if the firing rate of the excitatory neurons is approximately that of the inhibitory neurons for each point in time. Convince yourself that the network you have constructed is in the balanced state.

1.3. Using the equation for the synaptic input above, show that the expected synaptic input I^{syn} to any neuron is $\langle I^{\text{syn}}(t) \rangle = JK_E \left(r_E(t) - g\gamma r_I(t) \right)$ in the limit of infinite neurons but a fixed number of incoming connections.
Hint: Split the sum over neurons into an excitatory and an inhibitory part. In the limit of infinite neurons we may see the spike train of each neuron of a given type as independent and identically distributed. The expected firing rate of any neuron is then $\langle s_j^A(t) \rangle = \frac{1}{N} \sum_{i=1}^{N_A} s_i^A(t) \equiv r_A(t)$ where $A \in \{E, I\}$ is the neuron type.

The network is said to be *inhibition-dominated* if the contribution to the synaptic input of the inhibitory neurons is larger than that of the excitatory neurons.

1.4. Using the equation for the expected synaptic input to any neuron, argue what the value of g should be for the network to be inhibition-dominated. Use the same network configuration as Ex. 1.2 (apart from g) and $I_i^{\text{ext}}(t) = 0$ nA. Report (based on simulation) what happens to the mean firing rate of the neurons (both E and I, respectively) when the condition on g is not satisfied.

Finally, we will investigate more closely the interplay between excitation and inhibition. Specifically, what happens when the excitatory population receives external input but not the inhibitory one, and vice versa. For this, we take the following time-varying input current:

$$I_i^{\text{ext,E}}(t) = \begin{cases} I_0(1+\sin(\omega t)) & 0 \leq t < 500\text{ms} \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

$$I_i^{\text{ext,I}}(t) = \begin{cases} 0 & 0 \leq t < 500\text{ms} \\ I_0(1+\sin(\omega t)) & \text{otherwise} \end{cases} \tag{7}$$

using $I_0 = 10$ mV and $\omega = 25$ rad/s.

1.5. Simulate the network described above for $T = 1000$ ms using the same parameters as in Ex. 1.2 and produce the following two figures:

- A single rasterplot over time showing spikes from 100 excitatory and 25 inhibitory neurons, colour-coded by neuron type.
- A single plot of the two population firing rates (E/I) over time (same colourcoding), with activity combined in bins of 20 ms (and normalised accordingly).

1.6. Comment on what you observe in the previous exercise. Include in your answer the following points:

- Explain why only stimulating a single population (either E or I) breaks the balanced state, using Eq. (3) and Ex. 1.3.
- Explain how it is possible that a *positive* stimulation of the inhibitory neurons can lead to a *decrease* of their firing rate.

Ex 2. Field model of the cortical sheet

In the previous section we have constructed a network with two interacting populations (one E, one I). Now we will use this network as a single unit, which we will call one *E-I unit*, to construct a field model representing the cortical sheet. We construct our cortical sheet model by combining several of these E-I units spread out over many locations.
If N_{units} is the number of E-I units, it implies that there are N_{units} excitatory populations and also N_{units} inhibitory ones, making the total number of homogeneous populations equal to $2N_{\text{units}}$. We will allow interactions between any pair of populations, for which we define a population connectivity matrix W of size $2N_{\text{units}} \times 2N_{\text{units}}$. Note that the excitatory-inhibitory interactions in a single E-I unit still persist, and that unit-unit interactions happen *on top* of them. Specifically, we will be implementing a ring structure, for which we assign to each of our N_{units} E-I units a position x_α, equally spaced out over the range $[0, 1]$, with unit-unit interactions based on the distance between them.

2.1. Write down the two equations for the external input received by an E-I unit α as a function of the connectivity matrix W and the instantaneous population activities $r_E^\beta(t)$ and $r_I^\beta(t)$. Assume that the synaptic delay between two units is the same as between two neurons in the same population.

2.2. Implement a method for initialising and simulating the network as described above in your code, and verify for yourself that, with unit-unit interactions disabled, you retrieve the balanced state in each E-I unit.

Hint: It may be convenient to wrap your code from Exercise 1 into a class that defines a single E-I unit. Give this class a function `step(I_E, I_I)` that updates the dynamics of the unit for a single time step based on the external inputs to the E and I population, respectively.

For our cortical sheet model we would like to include short-range excitation and long-range inhibition. However, in cortex, it is commonly seen that inhibitory neurons only make short-range connections, and that it is the excitatory neurons that can make long-range connections. To implement the classical long-range inhibition under these constraints, we let each excitatory population excite the inhibitory populations that are far away (see figure below). Let σ be the range of excitatory-excitatory interactions. Then we define the interactions from unit β to unit α as

$$W^{E\leftarrow E}_{\alpha\beta} = W_0 f(x_\alpha, x_\beta) \tag{8}$$

$$W^{I\leftarrow E}_{\alpha\beta} = g\gamma W_0(1 - f(x_\alpha, x_\beta)) \tag{9}$$

$$W^{E\leftarrow I}_{\alpha\beta} = W^{I\leftarrow I}_{\alpha\beta} = 0 \tag{10}$$

with

$$f(x_\alpha, x_\beta) = \begin{cases} 1 & \text{if } d(x_\alpha, x_\beta) \leq \sigma \\ 0 & \text{otherwise} \end{cases} \tag{11}$$

where $d(x_\alpha, x_\beta)$ is the distance between x_α and x_β on the ring (including periodic conditions!). Since individual units already handle within-unit interactions, we exclude self-connections from the unit-unit interaction matrix, i.e. $W^{E\leftarrow E}_{\alpha\alpha} = 0$.

2.3. Write a method for generating the unit-unit connectivity matrix as described above, and implement it as a function `generate_unit_connectivity(Nunits, sigma, W0, g, gamma)`.

We take $N_{\text{units}} = 10$ and $\sigma = 0.2$, with parameters of each E-I unit identical to those in Ex. 1.2 (but each with randomly initialized sparse connectivity and starting potentials).

2.4. Simulate the network for $T = 200$ ms using $W_0 = 90$ pC. Produce a single 2D plot of the activity of the excitatory populations over time (axes: time (x) and unit id (y), colour: activity). You should observe the formation of a spontaneous bump of activity.

- Explain using the unit-unit connectivity matrix why the bump is (mostly) stable.
- At what time does the bump typically emerge? Does it always emerge centered around the same unit?

2.5. Repeat the simulation of the previous exercise for varying W_0. Report the minimum value of W_0 necessary to produce a bump.

2.6. *(bonus)* Could this field model be used for tracking the location of a stimulus, such as an animal's head direction? Apply a time-varying localised external stimulus to the excitatory populations in your network (for appropriate W_0) and compare the position of the bump to the stimulus location. For example, you may choose to simulate for $T = 1000$ ms, with fixed stimulus applied to one E population for 200 ms, then a different E population for the next 200 ms, etc.

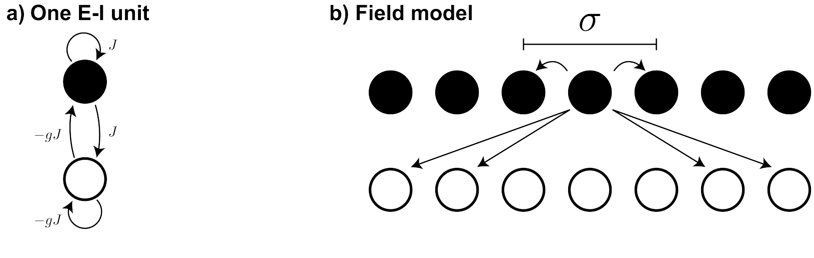

Figure 1: **a)** Single E-I unit consisting of two populations, one excitatory (top) and one inhibitory (bottom). **b)** Field model showing all outgoing connections from one central unit according to W.

Ex 3. Paradoxical response properties

With the cortical sheet model working we can now start to deeper investigate its properties, from the perspective of what an experimentalist might be able to do. Experimentalists are interested in finding the responses of single neurons to external stimuli to find what the neurons are selective to. Classically, neuron selectivity is reported as a *tuning curve* that shows how the mean firing rate of a neuron changes as a function of some external parameter controlling the stimulus (for example, stimulus orientation or location).

Here we will measure the tuning curves of the neurons in the network in response to two parameters, the stimulus location and size (width), which we will extract by applying localised stimuli to the excitatory neurons. Let's put the network in a state without spontaneous bump formation, and first stimulate a single unit with a step input

$$I^{\mathrm{ext,E}}(t) = \begin{cases} I_0 & t \geq 100 \text{ ms} \\ 0 & \text{otherwise} \end{cases} \tag{12}$$

3.1. Simulate the network from the previous exercise for $T = 200$ ms using $W_0 = 45$ pC. Apply the step input of strength $I_0 = 30$ nA to the excitatory population of unit 5 and produce a plot as in Ex. 2.4. Describe what you observe:

- What happens to the activity of the populations that are not directly stimulated?
- Why is it important that W_0 is below the critical value described in Ex. 2.5?

3.2. From your simulation, produce a plot of the mean excitatory activity (after stimulus onset) as a function of unit location. Argue why this plot may also be interpreted as the tuning curve of a single unit as a function of *stimulus* location (rather than unit location), making use of the radial symmetry of the experiment.

The more interesting part is what happens when we consider varying stimulus widths. We will repeatedly apply constant external stimuli (equivalently, stimulus onset at $t = 0$ ms) of varying width σ_{stim} centered around a location x_0. The constant input that the excitatory population of the unit at location x_α receives is

$$I_\alpha^{\mathrm{ext,E}}(t) = I_0 \exp\left(-\frac{d(x_\alpha, x_0)^2}{2\sigma_{\mathrm{stim}}^2}\right) \tag{13}$$

where $I_0 = 30$ nA. For each stimulus we will apply it for $T = 200$ ms and use the activity during the last 150 ms to determine the mean firing rate of populations of interest.

3.3. Choose an E-I unit that will be the unit that you want to investigate. Plot the mean firing rate of the excitatory population of this unit as a function of varying stimulus width σ_{stim} varying from 0 (=single unit) to 0.4 in steps of 0.04, each centered at the location of this unit.

You should observe that the firing rate of the population of interest at first increases for increasing stimulus width, but after some point actually *decreases*!

3.4. Using your knowledge of short-range excitation and long-range inhibition, how is it possible that the mean firing rate of a population can decrease with increasing stimulus width? And how does the width σ_{stim} after which activity decreases relate to the parameters of the connectivity?

There are two possible direct ways that can cause the firing rate of a population to decrease. It is either due to 1) increased inhibition, or 2) decreased excitation.

3.5. In your model, which of the two ways is the direct cause for the decreased firing rate of the excitatory population for wide stimuli? If in an experiment one would only be able to measure the direct cause, and not know the underlying circuit, what implications does your observed cause have for the underlying mechanism? Include in your answer how it relates to the inhibition-dominated state as discussed in Exercise 1.

Ex 4. Final Question

Please select **ONLY ONE** of the following three questions and answer it in half a page of free text. Conceptually this is the most challenging part of the miniproject! You need to think deeply and really understand the issues before you answer. You can choose the question that fits best your background and interests. For this final question you are **NOT** allowed to discuss with other teams of students working on the same project.

4.1. [**Theory-oriented**] Assuming that your project works well and has all the qualitative features that you expect it to have. Can you think of a special case of the model for which you can derive mathematically the exact answer, using the mathematical theories covered in class (for example field models)? How could you use this to PRECISELY test your program, to make sure that it is quantitatively correct? **Hint:** What type of mismatch would you expect if either the theory calculations or the model simulations are wrong by a factor of 2 at an unknown location? Test your model for quantitative correctness.

4.2. [**Coding-oriented**] The E-I unit in this miniproject was inspired by the paper from Brunel (2000). In the paper, they described their setup and parameters in the following rephrased paragraph (on the **next page**). What is the EXACT mapping between your code and the description in the paper? Where does it differ?

4.3. [**Biology-oriented**] This miniproject in computational neuroscience should have a link to neurobiology. Be creative and design an experiment that would enable you to test specific aspects of the model predictions. If necessary, think of additional modules or features that you would have to add to your model to make a comparison with experiments possible.

[Coding-oriented] Paper snippet from Brunel (2000)

We analyze the dynamics of a network composed of N integrate-and-fire (IF) neurons, from which N_E are excitatory and N_I inhibitory. Each neuron receives C randomly chosen connections from other neurons in the network, from which $C_E = \varepsilon N_E$ from excitatory neurons and $C_I = \varepsilon N_I$ from inhibitory neurons. It also receives C_{ext} connections from excitatory neurons outside the network. We consider a sparsely connected network with $\varepsilon = C_E/N_E = C_I/N_I \ll 1$. The depolarization $V_i(t)$ of neuron i ($i = 1, \dots, N$) at its soma obeys the equation

$$\tau \dot{V}_i(t) = -V_i(t) + RI_i(t) \tag{14}$$

where $I_i(t)$ are the synaptic currents arriving at the soma. These synaptic currents are the sum of the contributions of spikes arriving at different synapses (both local and external). These spike contributions are modeled as delta functions in our basic IF model:

$$RI_i(t) = \tau \sum_j J_{ij} \sum_k \delta\left(t - t_j^k - D\right) \tag{15}$$

where the first sum on the right hand side is a sum on different synapses ($j = 1, \dots, C + C_{\text{ext}}$), with postsynaptic potential (PSP) amplitude (or efficacy) J_{ij} , while the second sum represents a sum on different spikes arriving at synapse j, at time $t = t_j^k + D$ where t_j^k is the emission time of kth spike at neuron j, and D is the transmission delay. Note that in this model a single synaptic time D is present. For simplicity, we take PSP amplitudes equal at each synapse—that is, $J_{ij} = J > 0$ for excitatory external synapses, J for excitatory recurrent synapses (note the strength of external synapses is taken to be equal to the recurrent ones), and $-gJ$ for inhibitory ones. External synapses are activated by independent Poisson processes with rate ν_{ext}. When $V_i(t)$ reaches the firing threshold θ, an action potential is emitted by neuron i, and the depolarization is reset to the reset potential V_r after a refractory period τ_{rp} during which the potential is insensitive to stimulation. The external frequency ν_{ext} will be compared in the following to the frequency that is needed for a neuron to reach threshold in absence of feedback, $\nu_{\text{thr}} = \theta/(JC_E\tau)$. We first study the case in which inhibitory and excitatory neurons have identical characteristics, as in the model described above. In the following, using anatomical estimates for neocortex, we choose $N_E = 0.8N$, $N_I = 0.2N$ (80% of excitatory neurons). This implies $C_E = 4C_I$. We rewrite $C_I = \gamma C_E$ — that is, $\gamma = 0.25$. The number of connections from outside the network is taken to be equal to the number of recurrent excitatory ones, $C_{\text{ext}} = C_E$. We also choose $\tau_E = 20$ ms; $\theta = 20$ mV; $V_r = 10$ mV; $\tau_{\text{rp}} = 2$ ms.

Resources

The effect of surround inhibition on stimulus tuning is described in the paper Inhibitory stabilization of the cortical network underlies visual surround suppression *by H. Ozeki et al. (2009), and is also covered in* Section 18.2.4 *of the course book.*

You can learn more on balanced networks in the classic paper Dynamics of Sparsely Connected Networks of Excitatory and Inhibitory Spiking Neurons *by N. Brunel (2000). The synchronisation properties of the network are also discussed in* Section 13.4.2 *of the* Neuronal Dynamics *course book. A variant of the network is also covered in the* Python Exercises*.*